

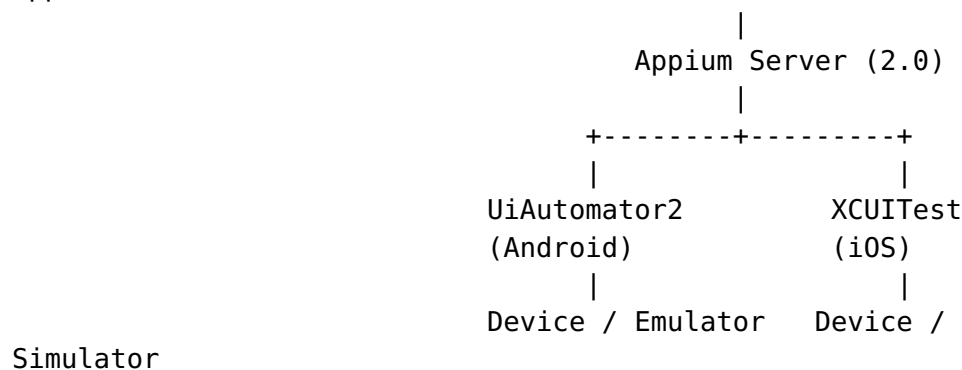
appium-adapter

Appium Adapter

The Appium adapter implements MobileAdapter using Appium 2.0, a cross-platform mobile testing server built on the W3C WebDriver protocol with Appium-specific extensions. It supports both Android and iOS through a unified interface.

Architecture

Go Challenge --> AppiumAdapter --> HTTP (W3C WebDriver + Appium Extensions)



The adapter communicates with the Appium server over HTTP using the W3C WebDriver session model. Appium extends the protocol with appium:-prefixed capabilities and mobile-specific endpoints (/appium/device/install_app, /appium/app/launch, etc.).

Prerequisites

1. **Appium Server 2.0** running and accessible over HTTP (default port 4723)
2. **Automation driver** installed: appium driver install uiautomator2 (Android) or appium driver install xcuitest (iOS)
3. **Android:** Android SDK, ADB in PATH, emulator or physical device connected

4. **iOS:** Xcode, Xcode Command Line Tools, WebDriverAgent built and deployed

AppiumCapabilities Reference

```
type AppiumCapabilities struct {
    PlatformName string `json:"platformName"`           //
        "Android" or "iOS"
    AutomationName string `json:"appium:automationName"` //
        "UiAutomator2", "XCUITest", "Espresso"
    DeviceName string `json:"appium:deviceName"`         //
        "emulator-5554", "iPhone 15"
    App string `json:"appium:app,omitempty"`             //
        Path to APK/IPA
    AppPackage string
        `json:"appium:appPackage,omitempty"`             // Android
        package
    AppActivity string
        `json:"appium:appActivity,omitempty"`             // Android launch
        activity
    BundleID string
        `json:"appium:bundleId,omitempty"`                 // iOS bundle ID
    PlatformVersion string
        `json:"appium:platformVersion,omitempty"`           // OS version
    NoReset bool
        `json:"appium:noReset,omitempty"`                   // Keep app data
    FullReset bool
        `json:"appium:fullReset,omitempty"`                 // Reinstall
        between sessions
    NewCommandTimeout int
        `json:"appium:newCommandTimeout,omitempty"`         // Idle
        timeout (seconds)
}
```

Capability Notes

- NoReset: true preserves app data and state between sessions (useful for login persistence)
- FullReset: true uninstalls and reinstalls the app before each session
- NewCommandTimeout prevents the Appium server from killing idle sessions
- When App is set, Appium auto-installs it on session creation

Constructor

```
adapter := userflow.NewAppiumAdapter(
    "http://localhost:4723",
    userflow.AppiumCapabilities{
        PlatformName: "Android",
        AutomationName: "UiAutomator2",
    },
)
```

```

        DeviceName:      "emulator-5554",
        AppPackage:      "com.example.app",
        AppActivity:     ".MainActivity",
    },
)

```

The constructor creates a `MobileConfig` from the capabilities and sets a 60-second HTTP timeout (longer than Selenium due to mobile device latency).

API Reference

Session Management

Session creation is deferred -- `initialize()` is called automatically by `LaunchApp`, `InstallApp`, or `RunInstrumentedTests` on first use. This avoids session timeouts when the adapter is created early.

IsDeviceAvailable

```
available, err := adapter.IsDeviceAvailable(ctx)
```

Queries `GET /status` on the Appium server. Returns true if `value.ready` is true in the response.

InstallApp, LaunchApp, StopApp

```
err := adapter.InstallApp(ctx, "/path/to/app.apk")
err = adapter.LaunchApp(ctx)
err = adapter.StopApp(ctx)
```

Uses Appium-specific endpoints: `/appium/device/install_app`, `/appium/app/launch`, `/appium/app/close`.

IsAppRunning

```
running, err := adapter.IsAppRunning(ctx)
```

Queries `/appium/device/app_state`. Returns true when the state value is ≥ 3 (running in background or foreground). States: 0 = not installed, 1 = not running, 2 = running in background (suspended), 3 = running in background, 4 = running in foreground.

TakeScreenshot

```
png, err := adapter.TakeScreenshot(ctx)
```

Uses the standard WebDriver GET /session/{id}/screenshot endpoint. Returns decoded PNG bytes.

Tap, SendKeys, PressKey

```
err := adapter.Tap(ctx, 500, 300)           // Tap at coordinates
err = adapter.SendKeys(ctx, "hello")        // Type into active
                                         element
err = adapter.PressKey(ctx, "KEYCODE_BACK") // Android key event
```

Tap uses W3C Actions API with pointer type "touch". SendKeys finds the active element first, then sends text via /element/{id}/value. PressKey uses Appium's /appium/device/press_keycode endpoint.

Supported key codes include: KEYCODE_HOME (3), KEYCODE_BACK (4), KEYCODE_MENU (82), KEYCODE_ENTER (66), KEYCODE_DEL (67), KEYCODE_VOLUME_UP (24), KEYCODE_VOLUME_DOWN (25), KEYCODE_POWER (26), KEYCODE_TAB (61), KEYCODE_SPACE (62), KEYCODE_ESCAPE (111), and media keys.

WaitForApp

```
err := adapter.WaitForApp(ctx, 30*time.Second)
```

Polls IsAppRunning every 500ms until the app is detected or the timeout expires.

RunInstrumentedTests

```
result, err := adapter.RunInstrumentedTests(ctx,
    "com.example.LoginTest")
```

Executes an instrument via Appium's execute_driver endpoint. Uses mobile: shell to run the Android instrumentation runner. Pass an empty string to run all instrumented tests.

Android Example

```
adapter := userflow.NewAppiumAdapter(
    "http://localhost:4723",
    userflow.AppiumCapabilities{
        PlatformName:  "Android",
        AutomationName: "UiAutomator2",
        DeviceName:    "Pixel_7_API_34",
        App:           "/builds/app-debug.apk",
        NoReset:       true,
    },
)
```

iOS Example

```
adapter := userflow.NewAppiumAdapter(  
    "http://localhost:4723",  
    userflow.AppiumCapabilities{  
        PlatformName:    "iOS",  
        AutomationName:  "XCUITest",  
        DeviceName:      "iPhone 15",  
        BundleID:        "com.example.MyApp",  
        PlatformVersion: "17.0",  
    },  
)
```

Docker Setup

```
services:  
  appium:  
    image: appium/appium:2.5  
    ports:  
      - "4723:4723"  
    volumes:  
      - /dev/bus/usb:/dev/bus/usb # USB passthrough for physical devices  
      - ./apps:/apps  
    environment:  
      - ANDROID_HOME=/opt/android-sdk  
    privileged: true
```

For emulator-based testing, use an Android emulator image:

```
services:  
  android-emulator:  
    image: budtmo/docker-android:emulator_14.0  
    ports:  
      - "5554:5554" # ADB  
      - "4723:4723" # Appium  
    environment:  
      - DEVICE=Samsung Galaxy S24  
      - APPIUM=true
```

Integration with MobileFlowChallenge

```
adapter := userflow.NewAppiumAdapter("http://appium:4723", caps)  
  
challenge := userflow.NewMobileFlowChallenge(  
    "CH-APPIUM-001",  
    "Appium Login Flow",  
    "Verify mobile login via Appium",  
    nil,
```

```

adapter,
userflow.MobileFlow{
    Name: "appium-login",
    Steps: []userflow.MobileStep{
        {Name: "tap-email", Action: "tap", X: 540, Y: 800},
        {Name: "type-email", Action: "send_keys", Text:
"user@test.com"},
        {Name: "tap-password", Action: "tap", X: 540, Y: 950},
        {Name: "type-password", Action: "send_keys", Text:
"secret123"},
        {Name: "tap-login", Action: "tap", X: 540, Y: 1100},
        {Name: "verify", Action: "screenshot"},
    },
},
)

```

Source Files

- Interface: pkg/userflow/adapter_mobile.go
- Implementation: pkg/userflow/appium_adapter.go
- Challenge template: pkg/userflow/challenge_mobile.go